

Ochrona Danych - laboratorium 4

sprawozdanie

Jakub Jaśków 268416

1 Cel ćwiczenia

Celem laboratorium było praktyczne zapoznanie się z działaniem algorytmu DES oraz z implementacją jego wybranych trybów. W szczególności celem było:

- analiza funkcji składowych algorytmu DES (HEXTOBIN, BINTOHEX, ks, cyfun, des, d_des),
- szyfrowanie i deszyfrowanie 8-bajtowych bloków danych,
- wprowadzenie obsługi ASCII dla tekstu jawnego i kluczy,
- realizacja szyfrowania plików w trybie DES ECB,
- realizacja szyfrowania plików w trybie DES CBC,
- porównanie poprawności poprzez weryfikację zgodności plików wejściowych i zdeszyfrowanych.

2 Opis algorytmu DES

Algorytm DES jest symetrycznym szyfrem blokowym operującym na blokach 64-bitowych. Wykorzystuje on klucz 64-bitowy (z czego 56 bitów stanowi klucz efektywny). DES składa się z:

- inicjalnej permutacji,
- 16 rund przetwarzania opartych na strukturze Feistela,
- końcowej permutacji odwrotnej.

Program korzysta z funkcji:

- **HEXTOBIN** — konwersja wartości hex na reprezentację binarną,
- **BINTOHEX** — konwersja bloków bitowych na cyfry hex,
- **ks** — generowanie 16 podkluczy,
- **cyfun** — funkcja rundowa,
- **des** — pełne szyfrowanie,
- **d_des** — deszyfrowanie (odwrotna kolejność rund).

3 Zadanie 1

Zadanie 1 polegało na analizie programów CIPH.CPP i DECIPH.CPP. Ponieważ pełna realizacja zagadnień zadania 1 została wykonana w ramach rozbudowanego programu w zadaniu 2, nie jest tu prezentowana oddzielnie.

4 Zadanie 2 – szyfrowanie i deszyfrowanie bloków ASCII i HEX

W zadaniu należało zmodyfikować program CIPH tak, aby możliwe było:

- szyfrowanie bloków złożonych z 8 znaków ASCII,
- deszyfrowanie bloków zapisanych jako 16 znaków hex,
- podawanie klucza w formie 8 znaków ASCII,
- niezależne wybieranie trybu szyfrowania lub deszyfrowania.

Powstał program działający zgodnie z poleceniem. Poniżej znajdują się screeny obrazujące działanie:

Wynik szyfrowania (screen 1)

```
mango@T14:~/sem2/0chrona_Danych/lab6/src$ ./ex2
*** PROGRAM N_CIPHER / N_DECIPHER ***

Choose mode:
  1 - N_CIPHER (encrypt 8 ASCII chars)
  2 - N_DECIPHER (decrypt 16 hex chars)
Type 1 or 2 + ENTER:
1

*** PROGRAM N_CIPHER ***

Type plaintext <8 ASCII characters + ENTER>:
Text ascii:
00000000
ssssssssss
ssssssss

Text hex:    7373737373737373
Text bin:    0111001101110011011100110111001101110011011100110111001101110011

Type Key <8 ASCII characters + ENTER>:
Key ascii:
00000000
4444444444
44444444

Key hex:    3434343434343434
Key bin:    0011010000110100001101000011010000110100001101000011010000110100

Ciphering is running...Done.

Cipher hex: 2a5ea167aff22a36
Cipher bin: 0010101001011110101000010110011110101111111100100010101000110110

Press ENTER
```

Rysunek 1: Szyfrowanie bloku 8 znaków ASCII

Wynik deszyfrowania (screen 2)

```
mango@T14:~/sem2/Ochrona_Danych/lab6/src$ ./ex2
*** PROGRAM N_CIPHER / N_DECIPHER ***

Choose mode:
 1 - N_CIPHER (encrypt 8 ASCII chars)
 2 - N_DECIPHER (decrypt 16 hex chars)
Type 1 or 2 + ENTER:
2

*** PROGRAM N_DECIPHER ***

Type ciphertext <16 hexadecimal characters [0..f] + ENTER>:
Text hex:
2a5ea167aff22a36
2a5ea167aff22a36
Text bin: 00101010010111101010000101100111101011111110010

Type Key <8 ASCII characters + ENTER>:
Key ascii:
00000000
4444444444
44444444

Key hex: 3434343434343434
Key bin: 001101000011010000110100001101000011010000110100

DeCiphering is running...Done.

Decipher ASCII: s s s s s s s s
Decipher hex: 7 3 7 3 7 3 7 3 7 3 7 3 7 3 7 3
Decipher bin: 0111001101110011011100110111001101110011011100

Press ENTER
```

Rysunek 2: Deszyfrowanie bloku hex do postaci ASCII

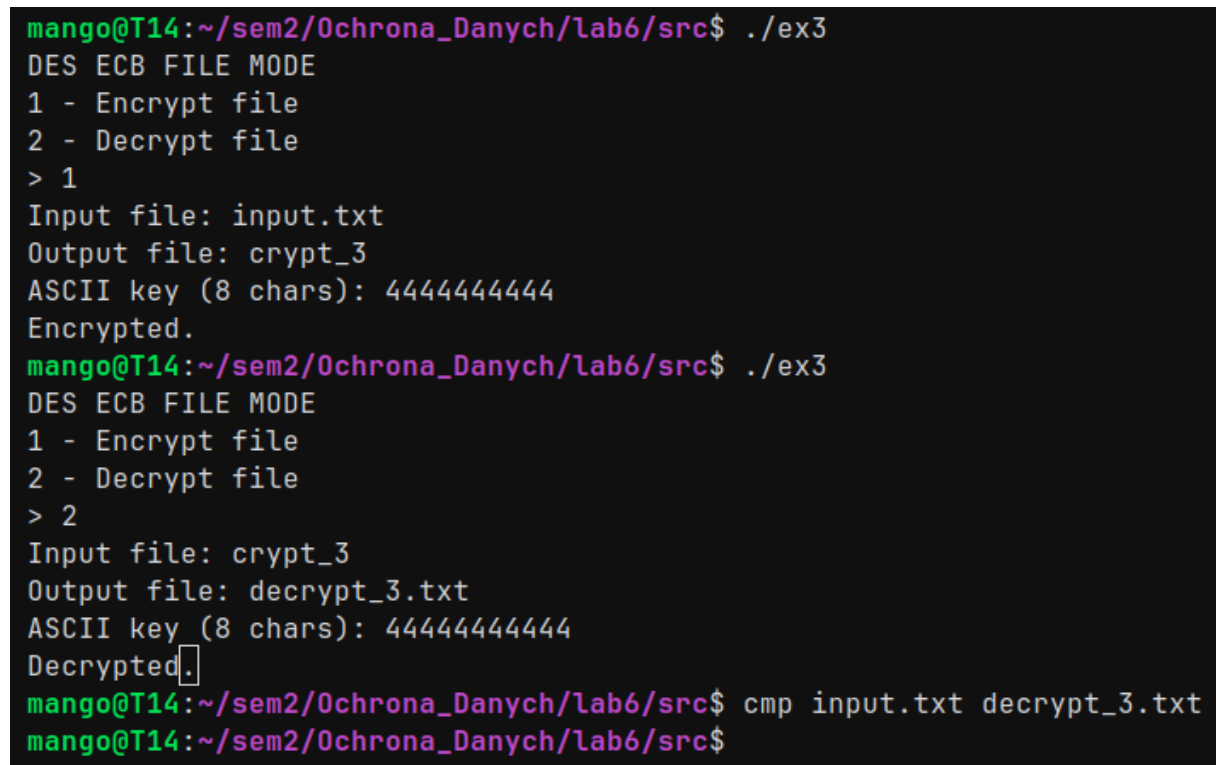
5 Zadanie 3 – szyfrowanie i deszyfrowanie plików w trybie DES ECB

Program został rozbudowany o obsługę plików. Szyfrowanie odbywa się blokami po 8 bajtów, a ostatni blok jest dopełniany losowymi wartościami. Dodatkowo na końcu dopisywany jest blok

zawierający informację o liczbie bajtów zajętych w ostatnim bloku.

Po deszyfrowaniu plik wynikowy jest identyczny z wejściowym, co potwierdzono poleceniem `cmp`.

Screen z działania programu (ECB)



```
mango@T14:~/sem2/0chrona_Danych/lab6/src$ ./ex3
DES ECB FILE MODE
1 - Encrypt file
2 - Decrypt file
> 1
Input file: input.txt
Output file: crypt_3
ASCII key (8 chars): 4444444444
Encrypted.
mango@T14:~/sem2/0chrona_Danych/lab6/src$ ./ex3
DES ECB FILE MODE
1 - Encrypt file
2 - Decrypt file
> 2
Input file: crypt_3
Output file: decrypt_3.txt
ASCII key (8 chars): 4444444444
Decrypted.
mango@T14:~/sem2/0chrona_Danych/lab6/src$ cmp input.txt decrypt_3.txt
mango@T14:~/sem2/0chrona_Danych/lab6/src$
```

Rysunek 3: Szyfrowanie i deszyfrowanie plików w trybie DES ECB

6 Zadanie 4 – szyfrowanie i deszyfrowanie plików w trybie DES CBC

Opracowano analogiczny program jak w zadaniu 3, jednak wykorzystujący tryb CBC (łańcuchowy). Tryb ten zwiększa bezpieczeństwo poprzez wykorzystanie wektora inicjalizującego IV oraz zależność kolejnych bloków od wyników poprzednich.

Po deszyfrowaniu plik wynikowy również był identyczny z plikiem wejściowym.

Screen z działania programu (CBC)

```
mango@T14:~/sem2/0chrona_Danych/lab6/src$ ./ex4
DES CBC FILE MODE
1 - Encrypt file (CBC)
2 - Decrypt file (CBC)
> 1
Input file: input.txt
Output file: crypt_4
ASCII key (8 chars): 444444444444
Encrypted (CBC).
mango@T14:~/sem2/0chrona_Danych/lab6/src$ ./ex4
DES CBC FILE MODE
1 - Encrypt file (CBC)
2 - Decrypt file (CBC)
> 2
Input file: crypt_4
Output file: decrypt_4.txt
ASCII key (8 chars): 444444444444
Decrypted (CBC).
mango@T14:~/sem2/0chrona_Danych/lab6/src$ cmp input.txt decrypt_3.txt
mango@T14:~/sem2/0chrona_Danych/lab6/src$
```

Rysunek 4: Szyfrowanie i deszyfrowanie plików w trybie DES CBC

7 Wnioski

- Implementacja algorytmu DES wymaga obsługi wielu operacji bitowych oraz permutacji.
- Rozszerzenie programu o obsługę ASCII pozwoliło uprościć wprowadzanie danych.
- Tryb ECB zachowuje poprawność przetwarzania, lecz nie zapewnia bezpieczeństwa strukturalnego danych.
- Tryb CBC eliminuje powtarzalność bloków kryptogramu, co zwiększa odporność szyfru.
- Przeprowadzone testy potwierdziły prawidłowe działanie programu — pliki zdeszyfrowane są identyczne jak wejściowe.